Comparative Study of Local Planners for Quadrupeds Using BARN Environments

Daniel Torres, Pranav A. Bhounsule

Department of Mechanical and Industrial Engineering, University of Illinois at Chicago, 842 W Taylor St, Chicago, IL 60607, USA. (e-mail: dtorre38@uic.edu; pranav@uic.edu)

Abstract: Integrating legged robots into motion planning frameworks is challenging because navigation and balance goals must be met simultaneously. We present a hierarchical framework that utilizes task-based motion control to reduce legged dynamics to a unicycle model for motion planning. Then, using the standardized Benchmark for Autonomous Robot Navigation (BARN) environments, we benchmark three established local planners: Dynamic Window Approach (DWA), Potential Fields (PF), and Model Predictive Control (MPC) and validate these approaches via simulation and hardware experiments. Our results demonstrate that a hierarchical framework offers a robust, computationally efficient pathway for deploying quadrupedal robots in obstacle-dense environments.

Keywords: Autonomous mobile robots, Legged robots, Hierarchical control, Path planning, BARN benchmark

1. INTRODUCTION

There has been substantial progress in the motion control of legged robots, enabling stable locomotion, but their broader usage requires them to be integrated into motion planning frameworks. Quadruped robots, unlike wheeled robots, pose a challenge because motion planning must be tightly coupled with balance control. In this work, we address this gap by benchmarking three local planning approaches: Dynamic Window Approach (DWA), Potential Fields (PF), and Model Predictive Control (MPC), on BARN environments in simulation and hardware.

There are broadly two types of planners, global and local. Global planners assume a complete map and compute a collision-free path in advance. Probabilistic Roadmaps (PRM), for example, build a tree over free space and search it with a method such as A-star Kavraki et al. (2002); Hart et al. (1968). This gives globally optimal solutions but is computationally intensive and requires re-planning when the environment changes. Local planners instead generate motion commands online from current sensor feedback, trading global optimality for computational efficiency and reactivity, though they can become trapped in local minima.

Prior work on motion planning for legged robots falls into integrated and hierarchical approaches. Integrated approaches solve planning and control simultaneously, enforcing dynamics, contact, and collision constraints directly, at high computational cost. Ren et al. (2023) combines a reinforcement-learning-based local planner with foothold adaptation, and Gaertner et al. (2021) uses nonlinear MPC with collision avoidance and joint optimization of forces and center-of-mass motion. Hierarchical approaches instead decouple planning and control. A low-level controller tracks velocity commands while a higher-level planner uses simplified models, as in Peng et al. (2025)'s Hybrid Zero dynamics controller with a 3D inverted pendulum planning model. This is computationally efficient but can request motions the controller cannot realize.



Fig. 1. Still-motion path of the robot in BARN Env. 000. The hardware trajectory is shown on top (obstacles are virtually enforced using a virtual camera), while the corresponding MuJoCo reconstruction from recorded hardware data with obstacles (white) is shown below. Video: http://tiny.cc/mecc26.

In this work, we adopt a hierarchical control framework. At the low level, we employ a computationally efficient task-level controller to ensure robust locomotion Bhounsule et al. (2023). This controller enables a reduced-order representation of the robot's center-of-mass dynamics using a unicycle model, which is then used for motion planning.

⋆ This material is based upon work supported by the National Science Foundation under Grant No. 2128568 to P. A. Bhounsule and by the National Science Foundation Graduate Research Fellowship Program under Grant No. 2236870 to D. Torres. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

Using the unicycle model, we study three widely used local planning approaches. DWA searches over admissible velocity commands while accounting for robot dynamics, enabling real-time obstacle avoidance Fox et al. (1997). PF methods generate motion by combining attractive and repulsive forces, providing simple and reactive behavior but suffering from local minima Khatib (1986). MPC formulates planning as a constrained optimization problem over a finite horizon, explicitly handling system dynamics and constraints at the cost of higher computational complexity Garcia et al. (1989).

Our contributions are as follows: (1) a hierarchical, computationally efficient control and planning framework for legged robots based on a reduced-order model; (2) an extensive comparison of three local planners: DWA, PF, and MPC; an evaluation using the BARN environments; and (3) experimental validation on a hardware testbed to demonstrate real-world performance.

2. METHODS

2.1 Hardware

The quadruped platform used in this paper is the Unitree A1 Unitree-Robotics (2022), which has four legs and 12 degrees of freedom (DOF). Each leg has a mass of 2 kg and the torso has a mass of 4.75 kg. The robot is equipped with joint encoders, a torso-mounted inertial measurement unit (IMU), and foot contact sensors. Joint torques are bounded between -33.5 and 33.5 Nm and are estimated via calibrated motor current sensing. Control runs on two systems. An external computer sends torque, position, and velocity commands at 1 kHz, while a proprietary onboard motor controller operates at 10 kHz and is not user-accessible. Communication is handled using Lightweight Communications and Marshalling (LCM) and User Datagram Protocol (UDP). Localization of the robot and obstacles is provided by a 12-camera OptiTrack system streaming over Wi-Fi at approximately 300 Hz.

2.2 Motion Control

Overview of control: The motion controller shown in Fig. 2 (a) coordinates the balance and locomotion of the quadruped and runs at 1 kHz. The legs are labeled as Front Right (FR), Front Left (FL), Rear Right (RR), and Rear Left (RL) as shown in Fig. 2 (b). For the trot gait, the combinations of FR-RL and FL-RR move together. The pair of legs cycles between stance and swing based on a fixed step time of 0.2 sec throughout this work. More details on the motion control are provided elsewhere Bhounsule et al. (2023).

Inputs to the motion planner: The motion planner (described in the next section) provides inputs to the motion controller at each step, operating at 5 Hz (step time is 0.2 sec). The planner specifies the fore-aft speed, $\dot{x}_{\text{ref}} = v$, and the turning rate, $\dot{\psi}_{\text{ref}} = \omega$. The following inputs are kept fixed in this work: lateral speed, $\dot{y}_{\text{ref}} = 0$; reference leg height, $z_{\text{ref}} = \bar{z}$; ground clearance; and torso roll and pitch, $\phi_{\text{ref}} = 0$ and $\theta_{\text{ref}} = 0$, respectively.

2.3 Motion Planning

Unicycle Model for planning: We assume that the motion controller presented earlier allows us to model the overall motion of the center of mass of the robot using the unicycle model. This model has the configuration variables (x, y, θ) and control inputs (v, ω). The continuous dynamics are $\dot{x} = v\cos\theta, \dot{y} = v\sin\theta, \dot{\theta} = \omega$. These equations are discretized using Euler integration with a time step of $h = 0.2$ s.

$$\begin{aligned} &x_{k+1} = x_k + hv_k\cos\theta_k, \quad y_{k+1} = y_k + hv_k\sin\theta_k, \\ &\theta_{k+1} = \theta_k + h\omega_k. \end{aligned} \tag{1}$$

Overview of planning: We use three local planning strategies: DWA, PF, and MPC. For each trial, the robot position (x_i, y_i) and the goal position (x_g, y_g) are known at all times. We model each obstacle as a circle with center (x_i, y_i) and radius r_i. In order to account for the robot size, we inflate each radius by an additional radius of r_s. This enables us to treat the robot as a point mass, thus simplifying the real-time calculations.

In the planner description, all gains and weighting parameters (e.g., k_a, k_r, k_ω, w_g, w_s, w_c, w_h, w_p, w_o) are positive scalar tuning parameters. All planners require parameter tuning, and gains are selected empirically and are consistent across all experiments.

Potential Field (PF): The potential field is defined as

$$U(\mathbf{p}) = U_{\text{att}}(\mathbf{p}) + \sum_{i=1}^{N} U_{\text{rep},i}(\mathbf{p}). \tag{2}$$

The attractive potential uses a conic-paraboloidal form with goal radius r_g:

$$U_{\text{att}} = \begin{cases} \dfrac{1}{2}k_a\|\mathbf{p}-\mathbf{p}_g\|^2, & \|\mathbf{p}-\mathbf{p}_g\| \le r_g, \\ k_a\, r_g\,\|\mathbf{p}-\mathbf{p}_g\|, & \text{otherwise.} \end{cases} \tag{3}$$

The repulsive potential is defined for obstacle distance $d_i = \|\mathbf{p} - \mathbf{q}_i\|$:

$$U_{\text{rep},i} = \begin{cases} \dfrac{1}{2}k_r\left(\dfrac{1}{d_i} - \dfrac{1}{d_0}\right)^2, & d_i \le d_0, \\ 0, & d_i > d_0. \end{cases} \tag{4}$$

The control inputs are derived from the artificial force $\mathbf{F} = -\nabla U$. The desired heading is

$$\theta_d = \text{atan2}(F_y,\, F_x). \tag{5}$$

The linear and angular velocities are computed as

$$v = s\,\|\mathbf{F}_{\text{att}}\|, \tag{6}$$

$$\omega = k_\omega\,\text{wrap}(\theta_d - \theta), \tag{7}$$

subject to saturation limits $v \in [v_{\min}, v_{\max}]$ and $\omega \in [-\omega_{\max}, \omega_{\max}]$. The scaling factor

$$s = \text{clip}\left(\frac{d_{\min} - r_s}{d_0},\, 0,\, 1\right) \tag{8}$$

reduces speed near obstacles, where $d_{\min}$ is the distance to the nearest obstacle and r_s is a safety radius.

Dynamic Window Approach (DWA): The dynamic window is defined as

$$\mathcal{DW} = \mathcal{V}_s \cap \mathcal{V}_d, \tag{9}$$

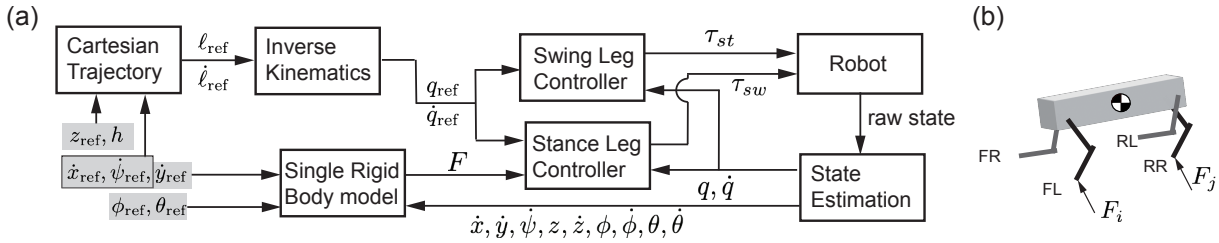


Fig. 2. (a) Block diagram of the robot controller. A state machine is used to program the robot controller logic. The inputs to the state machine are shown in the box with gray highlight, but only two inputs, the linear speed in fore-aft direction $\dot{x}_{\text{ref}} = v_x = v$ and yaw speed $\dot{\phi}_{\text{ref}} = \omega$ are used for control in this paper. (b) The single rigid body model is used for regulating forces/torque on the body from the stance leg. Details are provided in Bhounsule et al. (2023).

where $\mathcal{V}_s = [v_{\min}, v_{\max}] \times [-\omega_{\max}, \omega_{\max}]$ and

$$\mathcal{V}_d = [v \pm \dot{v}_{\max}\Delta t] \times [\omega \pm \dot{\omega}_{\max}\Delta t]. \tag{10}$$

Each candidate $(v, \omega) \in \mathcal{DW}$ is forward simulated using Eq. (1). A trajectory τ is admissible if the robot can stop before collision:

$$\frac{v^2}{2\dot{v}_{\max}} \leq d_{\min}(\tau) - r_s, \tag{11}$$

where $d_{\min}(\tau)$ is the minimum distance to obstacles.

Admissible trajectories are ranked by

$$\begin{aligned} J(v, \omega) = w_g \, |\text{wrap}(\theta_h - \theta_{\text{goal}})| \\ + w_s \frac{v_{\max} - v}{v_{\max}} + w_c \, C_{\text{obs}}(\tau), \end{aligned} \tag{12}$$

where x_h, y_h, and θ_h denote the robot's position and heading at the end of the forward-simulated trajectory τ, and $\theta_{\text{goal}} = \text{atan2}(y_g - y_h, \, x_g - x_h)$. The optimal control is selected as

$$(v^*, \omega^*) = \arg\min_{(v,\omega)\in\mathcal{DW}} J(v, \omega). \tag{13}$$

Model Predictive Control (MPC): The MPC planner optimizes a control sequence $\mathbf{u}_{0:N-1}$ over a horizon of length N, with predicted states generated using Eq. (1). Let $\mathbf{p}_k = [x_k, y_k]^\top$ and $\mathbf{g} = [x_g, y_g]^\top$. The cost function is

$$\begin{aligned} J = \sum_{k=1}^{N} \Big[\frac{w_g}{d_0^2} \|\mathbf{p}_k - \mathbf{g}\|^2 + w_h\big(1 - \cos\theta_{kg}\big) - w_p v_{k-1} \cos\theta_{kg} \Big] \\ + \sum_{k=0}^{N-1} \mathbf{u}_k^\top R \, \mathbf{u}_k + \sum_{k=1}^{N} w_o \, \phi(d_k), \end{aligned} \tag{14}$$

where $\theta_{kg} = \theta_k - \theta_g$, $\theta_g = \text{atan2}(y_g - y_k, \, x_g - x_k)$ and $d_k = \min_i \|\mathbf{p}_k - \mathbf{q}_i\|$.

The matrix $R = \text{diag}(w_v/v_{\max}^2, \;\; w_\omega/\omega_{\max}^2)$ penalizes control effort. The function $\phi(d)$ is a smooth obstacle penalty that increases sharply within a safety radius.

The optimization is subject to box constraints

$$0 \leq v_k \leq v_{\max}, \quad -\omega_{\max} \leq \omega_k \leq \omega_{\max}, \tag{15}$$

A rollout is rejected if any predicted state violates a collision clearance threshold. The MPC optimization problem is solved using the SLSQP solver from the `scipy.optimize` module with a horizon of length 5.

2.4 BARN: Standardized Planning Environments

The Benchmark for Autonomous Robot Navigation (BARN) is a standardized dataset of 300 cluttered, narrow-passage environments designed to evaluate navigation performance under challenging local feasibility constraints. It includes precomputed difficulty metrics such as distance to closest obstacle, average visibility, dispersion, characteristic dimension, and tortuosity Perille et al. (2020).

The BARN dataset, based on the Clearpath Robotics Jackal unmanned ground vehicle (UGV), utilizes the same environments for the Unitree A1 robot due to their similar circumscribed diameters. To enhance feasibility in these constrained settings, obstacle radii are reduced by 25% (0.075 m) across simulation and hardware experiments.

For each environment, let $L_i^\star$ denote the reference path length. The optimal traversal time (OT_i) is defined as

$$OT_i = \frac{L_i^\star}{v_{\max}}, \tag{16}$$

where $v_{\max} = 0.5$ m/s is the robot's maximum speed.

Let AT_i denote the actual traversal time. The BARN score is defined as Xiao et al. (2026)

$$s_i = 1_i^{\text{success}} \times \frac{OT_i}{\text{clip}(AT_i, \, 2OT_i, \, 8OT_i)} \tag{17}$$

where the indicator function 1^{success} evaluates to 1 if the robot reaches the navigation goal without any collision and evaluates to 0 otherwise. This bounds the score to $[0.125, 0.5]$ for successful runs and assigns 0 to failures (i.e., collisions or timeouts). Timeouts are defined as trials exceeding 60 seconds of simulation or experimental time.

Perception Model: We assume a virtual range sensor with a 90° field of view and 1° angular resolution, returning distance measurements at each bearing angle up to a maximum range of 2.0 m. Measurements at or beyond this range are set to the maximum value and discarded. Each valid measurement is converted into a point $\mathbf{q}_i$ from its bearing and range, forming the set of perceived obstacle points.

The sensor model is identical in simulation and hardware. Both generate measurements by ray casting against known obstacle geometry rather than a physical sensing device, allowing controlled and repeatable evaluation.

2.5 Execution and Compute Setup

All experiments were conducted on a desktop with an Intel Core i7-9700K CPU (3.6 GHz, 8 cores) running Ubuntu 20.04. Low-level control was implemented in C/C++,

while local planners were implemented in Python. Each environment–planner pair was evaluated independently, with runs parallelized across 7 worker processes. All planners were required to operate within a 0.2 s control loop, consistent with the hardware deployment. Compute-time statistics are reported per planner and are independent of parallel execution.

3. RESULTS AND DISCUSSION

3.1 Simulation Experiments

We compare the performance of DWA, MPC, and PF across all 300 BARN environments (900 total runs) to assess robustness under consistent obstacle scaling and varying environment complexity.

DWA and PF achieve identical goal rates, each reaching the goal in 209 environments (70%), while MPC performs worst with 152 successes (51%), as shown in Table 1. Timeout rates are similar across planners, though slightly higher for MPC (26, 9%) compared to DWA (21, 7%) and PF (17, 6%). A similar trend is observed in collision rates, with DWA and PF at 70 (23%) and 74 (25%), respectively, and MPC higher at 122 (41%). These results indicate reduced robustness of MPC in cluttered environments under the current formulation and tuning. The BARN score reflects these trends, with DWA and PF achieving identical performance 0.35 ± 0.23, while MPC attains a lower average score (0.25 ± 0.25), consistent with its reduced success rate.

Among successful runs, MPC produces more efficient trajectories, with lower traversal time (12.64 ± 6.67 s) and shorter path length (4.29 ± 0.23 m) than DWA (20.38 ± 11.79 s, 5.82 ± 2.24 m) and PF (21.10 ± 10.09 s, 5.53 ± 1.52). This highlights a trade-off between efficiency and robustness. Minimum clearance is similar across planners (0.09–0.11 m), indicating comparable safety margins.

Table 1. **Simulation:** Planner performance and compute-time statistics across all environments. Success metrics are computed over all runs, while traversal time, path length, and minimum clearance are reported over successful runs only (mean ± standard deviation).

Metric	**DWA**	**MPC**	**PF**
Goal rate	209 (70%)	152 (51%)	209 (70%)
Collision rate	70 (23%)	122 (41%)	74 (25%)
Timeout rate	21 (7%)	26 (9%)	17 (6%)
BARN score	0.35 ± 0.23	0.25 ± 0.25	0.35 ± 0.23
Traversal time [s]	20.38 ± 11.79	12.64 ± 6.67	21.10 ± 10.09
Path length [m]	5.82 ± 2.24	4.29 ± 0.23	5.53 ± 1.52
Minimum clearance [m]	0.09 ± 0.09	0.09 ± 0.10	0.11 ± 0.10
Mean [ms]	10.27 ± 0.88	25.59 ± 26.06	0.18 ± 0.04
p95 [ms]	13.57	93.16	0.20
Max [ms]	21.46	191.76	1.12

Planner performance per environment is shown in Fig. 3, where only successful simulations receive a BARN score, with higher values indicated by warmer colors on the color bar. Collisions cluster more in higher-index, more difficult environments, illustrated by two representative environments and their planner trajectories in Fig. 4. Environment 273 is substantially more constrained than Environment 000, with lower average clearance (1.30 vs. 3.68), lower visibility (3.39 vs. 6.85), and a smaller characteristic dimension (1.29 vs. 5.11), along with higher dispersion (4.71 vs. 3.44). In the relatively open Environment 000, all three planners reached the goal. In contrast, Environment 273 produces divergent behaviors. MPC fails to reach the goal, colliding shortly after initiation. DWA reaches the goal but requires an immediate turn to avoid a nearby obstacle. PF maintains greater clearance throughout, resulting in a longer but safer trajectory.

The computational cost of each planner is summarized in Table 1. DWA and PF exhibit low and consistent computation times, with mean runtimes of 10.27 ms and 0.18 ms, and minimal variance. MPC shows high variability. Its average runtime is 25.59 ms, but its p95 latency reaches 93.16 ms and its maximum reaches 191.76 ms. It is often fast but can incur substantial latency spikes, which is problematic for real-time deployment since the worst-case delays approach the 0.2 s control loop limit and could degrade or destabilize behavior on hardware. MPC may therefore need additional safeguards or simplification to ensure consistent real-time performance, unlike DWA and PF.

Overall, these results highlight a trade-off between efficiency and robustness. MPC produces more efficient, but computationally expensive, trajectories when successful. DWA and PF favor reliability across a wider range of environments. Performance may also be influenced by the sensitivity of each planner to parameter tuning, particularly in cluttered environments.

3.2 Hardware Experiments

To evaluate real-world performance, the planners are deployed on hardware in two environments, BARN Env. 042 and 000. Hardware experiments assess execution fidelity and repeatability under real-time constraints and noise, complementing the simulation results. Two experiments were conducted: (1) velocity tracking performance in a simple environment to evaluate the robot's ability to follow commanded velocities, and (2) repeated trials in a single environment to assess planner consistency and robustness on hardware. Physical obstacles were not placed in either experiment. BARN environments contain dozens of obstacles per course, making physical placement impractical at this scale, and the platform carried no onboard perception system, so obstacles would not have been sensed regardless. Obstacle information came instead from the virtual perception model in Section 2.4.1, held identical between simulation and hardware.

Velocity Tracking Performance Velocity tracking performance is evaluated to isolate the effects of control execution from planning. A simple environment (BARN Env. 042) is chosen due to its direct path to the goal. The robot is commanded to follow planner-generated velocity inputs, and the resulting tracking error is shown in Fig. 5. The results indicate low tracking error across all planners, with mean absolute error (MAE) of 0.04 m/s and 0.01 rad/s for DWA, 0.04 m/s and 0.02 rad/s for MPC, and 0.05 m/s and 0.03 rad/s for PF. These errors are small relative to the commanded velocities and are attributed to system dynamics and sensing noise. Overall,

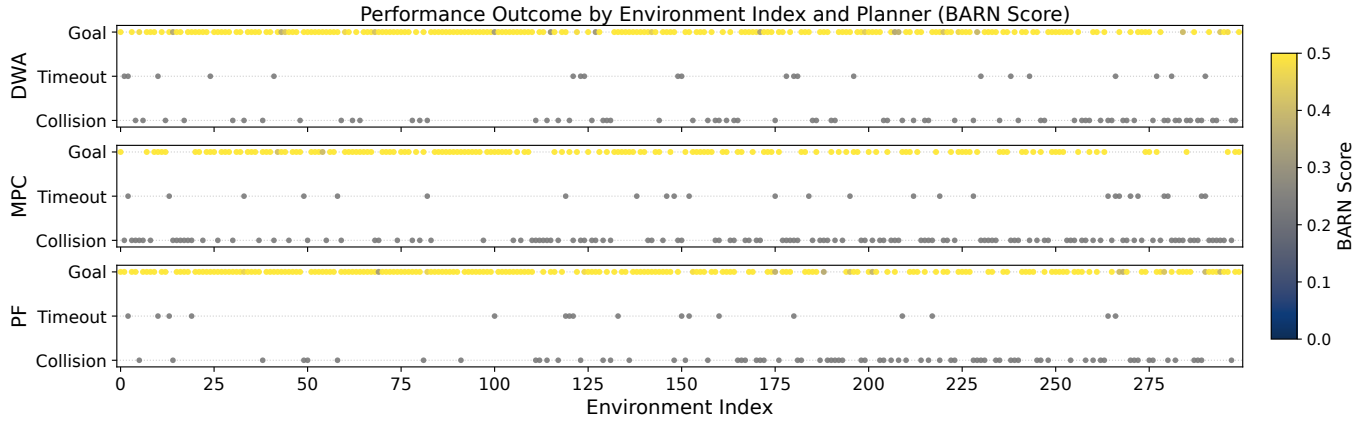


Fig. 3. **Simulation:** Performance outcomes across BARN environments for all planners. Each point is one run; vertical position shows the outcome (collision, timeout, goal), and color indicates BARN score for successful runs.

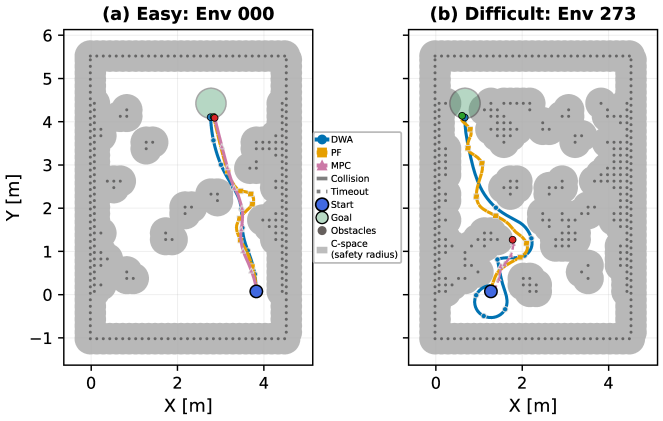


Fig. 4. **Simulation:** Representative BARN environments: (a) sparse with wide passages and (b) dense with narrow corridors. Dashed lines indicate collision.

the control system demonstrates sufficient accuracy to execute planner commands, suggesting that performance differences observed in more complex environments are primarily due to planning behavior rather than control limitations.

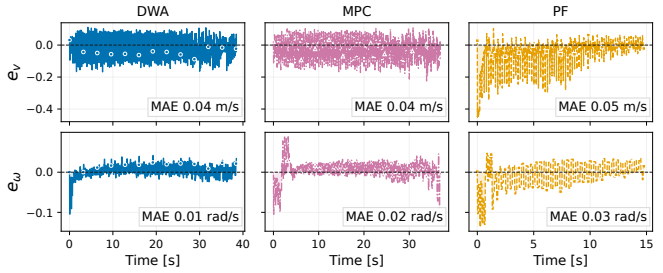


Fig. 5. **Hardware:** Velocity tracking error in BARN Env. 042 for (a) DWA, (b) MPC, and (c) PF. Each panel shows the difference between estimated and commanded linear and angular velocity, with mean absolute error (MAE) annotated for reference.

Single Environment Validation To evaluate planner consistency on hardware, each planner was tested in a single environment (BARN Env. 000) across five repeated trials. This experiment assessed the repeatability of planner behavior under real-world conditions. Obstacle information is provided through the virtual perception model described in Section 2.4.1, while robot pose and goal location are obtained via motion capture.

Figure 1 illustrates a representative DWA run, showing both the physical hardware trajectory and the corresponding MuJoCo reconstruction. While the hardware environment contains no physical obstacles, the reconstructed trajectory overlays the virtual obstacle map used for planning, providing a clear visualization of how the robot navigates the environment using the virtual perception model.

All three planners successfully completed BARN Env. 000 across all five trials (100%). Despite identical success rates, clear differences in trajectory efficiency and consistency emerge. PF achieves the highest BARN score of 0.50 ± 0.00 and the lowest traversal time (17.10 ± 0.37 s), while maintaining comparably short paths (4.35 ± 0.03 m). This indicates highly consistent and efficient behavior across repeated runs. DWA exhibits moderate performance, achieving a lower BARN score (0.28 ± 0.02) and significantly higher traversal time (32.54 ± 2.23 s), while producing slightly shorter paths (4.31 ± 0.04 m). In contrast, MPC performs worst across these metrics, with the lowest BARN score (0.26 ± 0.01), highest traversal time (35.10 ± 2.05 s), and longest path length (4.58 ± 0.24 m). In addition to reduced efficiency, MPC exhibits greater variability in trajectory selection across repeated trials, as shown in Fig. 6. While DWA and PF reproduce consistent and repeatable paths, MPC demonstrates noticeable variation between runs, indicating increased sensitivity to noise.

Table 2. **Hardware:** Performance and compute-time statistics for BARN Env. 000, averaged over five runs.

Metric	**DWA**	**MPC**	**PF**
Goal rate	5 (100%)	5 (100%)	5 (100%)
BARN score	0.28 ± 0.02	0.26 ± 0.01	0.50 ± 0.00
Traversal time [s]	32.54 ± 2.23	35.10 ± 2.05	17.10 ± 0.37
Length [m]	4.31 ± 0.04	4.58 ± 0.24	4.35 ± 0.03
Mean [ms]	10.52 ± 0.12	19.19 ± 9.06	0.19 ± 0.00
p95 [ms]	12.47	44.30	0.26
Max [ms]	21.67	142.77	0.32

The compute-time statistics further highlight these differences. DWA and PF maintain low and consistent runtimes, whereas MPC exhibits significantly higher variability, with

a mean runtime of 19.19 ± 9.06 ms, a p95 of 44.30 ms, and a maximum of 142.77 ms. These values remain within the 0.2 s control loop, but the presence of latency spikes is consistent with the variability observed in trajectory execution.

Overall, the hardware results reinforce trends observed in simulation while highlighting the impact of real-world variability. PF achieves the most consistent and efficient performance in this environment. DWA provides stable but less efficient behavior, while MPC exhibits reduced efficiency and increased variability under the current formulation.

A possible explanation for these differences lies in the state information required by each planner. DWA and MPC rely on estimates of the robot's current linear and angular velocities for control computation, making them more sensitive to noise and estimation errors in these quantities. In contrast, the PF method primarily computes control actions based on position information relative to the goal and obstacles, and does not depend on velocity feedback. As a result, PF is less affected by velocity estimation noise, which may contribute to its improved consistency and efficiency in hardware experiments.

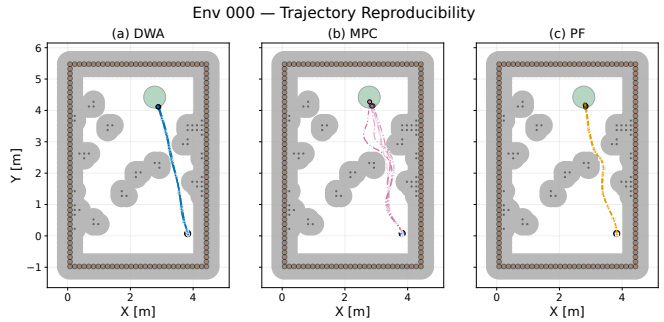


Fig. 6. **Hardware:** Trajectory reproducibility across 5 runs on BARN Env. 000 for (a) DWA, (b) MPC, and (c) PF. All runs successfully reached the goal, with PF producing the most consistent paths and MPC exhibiting the highest path-length variability.

4. CONCLUSION

This work presented a reduced-order planning framework and evaluated three local planners: DWA, MPC, and PF. These planners were evaluated using the BARN environments, providing a standardized and validated benchmark for navigation performance. The hierarchical structure itself performed as intended: reducing the robot to a unicycle model at the planning level kept all three planners computationally tractable while running unmodified on the physical quadruped, directly supporting the framework's hardware-validation goal.

Results show that while MPC can produce more efficient trajectories, it exhibits reduced robustness and increased computational variability under the current formulation. In contrast, DWA and PF provide more consistent performance, with PF demonstrating the strongest hardware performance due to its reduced dependence on velocity estimation.

These findings highlight the importance of balancing trajectory optimality with robustness and computational consistency when selecting local planners for real-time robotic systems. Future work will extend evaluations to real-world perception using onboard sensing and physical obstacles. We will also improve the robustness and real-time performance of optimization-based planners, such as MPC, and investigate adaptive, online gain-tuning methods for varying environments.

REFERENCES

Bhounsule, P.A., Torres, D., Hinojosa, E.H., and Alaeddini, A. (2023). Task-level control and poincaré map-based sim-to-real transfer for effective command following of quadrupedal trot gait. In *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*, 1–8. doi:10.1109/Humanoids57100.2023.10375183.

Fox, D., Burgard, W., and Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE Robotics and Automation Magazine*, 4(1), 23–33. doi:10.1109/100.580977.

Gaertner, M., Bjelonic, M., Farshidian, F., and Hutter, M. (2021). Collision-free mpc for legged robots in static and dynamic scenes. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 8266–8272. IEEE.

Garcia, C.E., Prett, D.M., and Morari, M. (1989). Model predictive control: Theory and practice—a survey. *Automatica*, 25(3), 335–348.

Hart, P.E., Nilsson, N.J., and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2), 100–107.

Kavraki, L.E., Svestka, P., Latombe, J.C., and Overmars, M.H. (2002). Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE transactions on Robotics and Automation*, 12(4), 566–580.

Khatib, O. (1986). Real-time obstacle avoidance for manipulators and mobile robots. *The international journal of robotics research*, 5(1), 90–98.

Peng, C., Paredes, V., Castillo, G.A., and Hereid, A. (2025). Real-time safe bipedal robot navigation using linear discrete control barrier functions. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 14903–14909. IEEE.

Perille, D., Truong, A., Xiao, X., and Stone, P. (2020). Benchmarking metric ground navigation. In *2020 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 116–121. IEEE Press. doi:10.1109/SSRR50563.2020.9292572. URL `https://doi.org/10.1109/SSRR50563.2020.9292572`.

Ren, J., Dai, Y., Liu, B., Xie, P., and Wang, G. (2023). Hierarchical vision navigation system for quadruped robots with foothold adaptation learning. *Sensors*, 23(11). doi:10.3390/s23115194. URL `https://www.mdpi.com/1424-8220/23/11/5194`.

Unitree-Robotics (2022). A1 more dexterity, more possibilities. `https://www.unitree.com/products/a1/`.

Xiao, X., Xu, Z., Ghani, S.A., Datar, A., Song, D., Stone, P., Mazen, A., Yazdipaz, K., Mateyaunga, I., Faied, M., et al. (2026). Autonomous ground navigation in highly constrained spaces: Lessons learned from the fourth barn challenge at icra 2025. *IEEE Robotics & Automation Magazine*, 33(1), 186–196.